EE451 PHW2
Rafael Wang
rafaelwa@usc.edu
USC ID: 6189106477

For the entire PHW, the test platform used was the USC CARC Discovery cluster. The allocated compute node had the configuration --cpus-per-task=64 --mem=8G. The number of runs performed for each problem was 1. In terms of other necessary variables, the compiler used was g++.

For both p1a and p1b, the matrix size is 4K x 4K.
For p1a, the execution time for p x p = 1 is shown in the output below:

```
bash-4.4$ ./p1a 1
Execution time = 758.536 sec
C[100][100]=8.79616e+08
```

The execution time for p x p = 4 is shown in the output below:

```
bash-4.4$ ./p1a 2
Execution time = 339.012 sec
C[100][100]=8.79616e+08
```

The execution time for p x p = 16 is shown in the output below:

```
bash-4.4$ ./p1a 4
Execution time = 58.9873 sec
C[100][100]=8.79616e+08
```

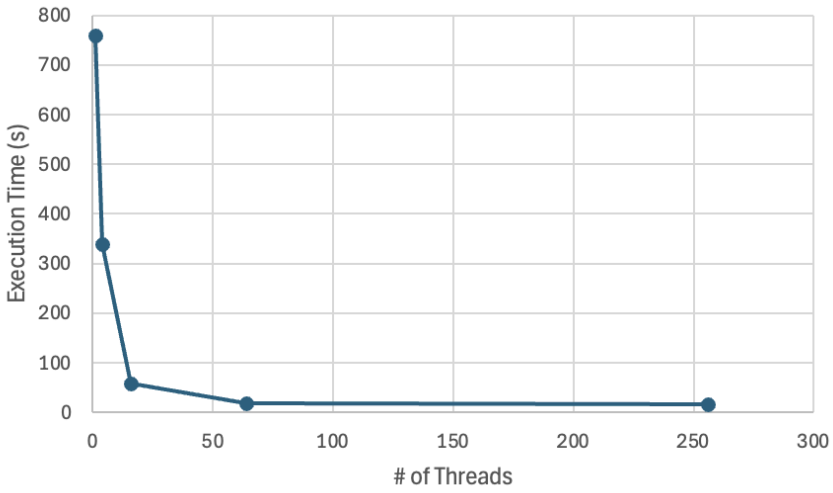
The execution time for p x p = 64 is shown in the output below:

```
bash-4.4$ ./p1a 8
Execution time = 17.4669 sec
C[100][100]=8.79616e+08
```

The execution time for p x p = 256 is shown in the output below:

```
bash-4.4$ ./p1a 16
Execution time = 17.1795 sec
C[100][100]=8.79616e+08
```

The diagram showing execution times under various p is shown below:



In terms of observations, there is a sharp improvement in execution time as the number of threads is increased from 1 to 16. However, past that, the improvement becomes minimal, and it eventually plateaus.

In terms of partitioning, matrix C is partitioned such that each thread is responsible for computing a block of matrix C of size n/p by n/p. Namely, thread (i,j) computes from row i*(n/p) to i*(n/p)+(n/p)-1 and from column j*(n/p) to j*(n/p)+(n/p)-1.

For p1b, the execution time for p x p = 1 is shown in the output below:

```
bash-4.4$ ./p1b 1
Execution time = 762.691 sec
C[100][100]=8.79616e+08
```

The execution time for p x p = 4 is shown in the output below:

```
bash-4.4$ ./p1b 2
Execution time = 199.171 sec
C[100][100]=8.79616e+08
```

The execution time for p x p = 16 is shown in the output below:

```
bash-4.4$ ./p1b 4
Execution time = 51.4315 sec
C[100][100]=8.79616e+08
```

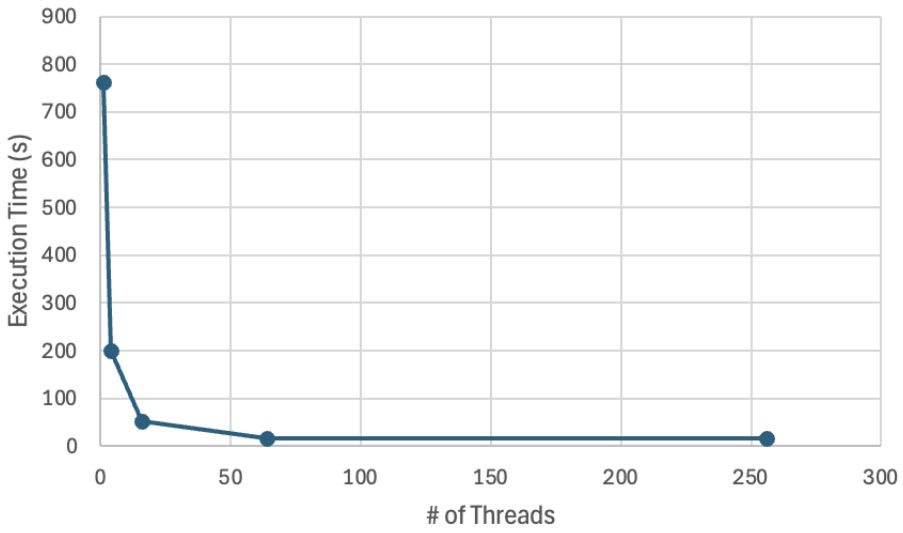
The execution time for p x p = 64 is shown in the output below:

```
[bash-4.4$ ./p1b 8
 Execution time = 15.6312 sec
 C[100][100]=8.79616e+08
```

The execution time for p x p = 256 is shown in the output below:

```
[bash-4.4$ ./p1b 16
 Execution time = 15.2888 sec
 C[100][100]=8.79616e+08
```

The diagram showing execution times under various p is shown below:



In terms of observations, there is once again a sharp improvement in terms of execution time as the number of threads is increased from 1 to 16. As before, the improvement begins plateauing as the number of threads is eventually increased up to 256.

In terms of partitioning, matrix A is partitioned into blocks of size n/p by n/p. Namely, thread (i,j) is assigned to a partition of matrix A from row i*(n/p) to i*(n/p)+(n/p)-1 and from column j*(n/p) to j*(n/p)+(n/p)-1. Each thread uses its partition A(i,j) and multiplies with the corresponding B(j, k) to accumulate C(i, k), for k = 0, 1, … n-1.

The execution time of 1.2 versus the execution time of 1.1 reveals that they have the same general trends, but the execution times for 1.2 are better than those for 1.1. This is most prevalent at 4 threads. A potential reason for this difference is that in 1.2, cache locality is better utilized as both matrices A and B are accessed row by row. On the other hand, in 1.1, matrix B is accessed in column-major fashion, but multi-dimensional arrays in C++ are stored in row-major layout.

For p2a, the execution time for p = 1 is shown in the output below:

```
bash-4.4$ ./p2a 1
Execution time: 1.98893 seconds
```

The execution time for p = 2 is shown in the output below:

```
bash-4.4$ ./p2a 2
Execution time: 1.23361 seconds
```

The execution time for p = 4 is shown in the output below:

```
bash-4.4$ ./p2a 4
Execution time: 1.09634 seconds
```

The execution time for p = 8 is shown in the output below:

```
bash-4.4$ ./p2a 8
Execution time: 0.796852 seconds
```

For p2b, the execution time for p = 1 is shown in the output below:

```
bash-4.4$ ./p2b 1
Execution time: 1.92657 seconds
```

The execution time for p = 2 is shown in the output below:

```
bash-4.4$ ./p2b 2
Execution time: 1.21111 seconds
```

The execution time for p = 4 is shown in the output below:

```
bash-4.4$ ./p2b 4
Execution time: 0.684447 seconds
```

The execution time for p = 8 is shown in the output below:

```
bash-4.4$ ./p2b 8
Execution time: 0.761162 seconds
```

Comparing the serial, 2.1, and 2.2 execution times, the serial version without using Pthreads saw an execution time of 1.00773 seconds, while the 2.1 execution times improved from 1.98893 seconds to 0.796852 seconds, and the 2.2 execution times improved from 1.92657 seconds to 0.761162 seconds. Thus, the serial version without Pthreads ran about as quickly as 2.1 for p=4 before getting outperformed by p=8, and ran slightly faster than 2.2 for p=2 before getting outperformed by p=4 and p=8. This may be because of the overhead from spawning threads and parallelizing code. In terms of observations, execution times for 2.1 generally improved as the number of threads increased, while in 2.2, p=4 was the optimal number of threads before execution time got worse at p=8. Furthermore, 2.2's best execution time was better than 2.1's best execution time, which may be because threads were not being created every iteration.

The output image output1 is shown below:

The output image output2 is shown below: